

CAMADA FÍSICA DA COMPUTAÇÃO · INS PER

Projeto 3

Transferência de Arquivos por Datagramas

Como dois Arduinos conversam por UART e recriam, em miniatura, os mesmos truques que a internet usa para não perder dados.

Guia de leitura rápida: depois de ler este documento inteiro, você deve conseguir explicar o projeto do início ao fim usando só suas próprias palavras — sem precisar olhar o código.

1 Qual é o problema?

Por que não basta simplesmente "mandar o arquivo"

Dois computadores estão ligados por um fio serial (UART) simples e lento, através de Arduinos usados só como conversores USB-serial. Esse fio **não garante nada**: não avisa se um byte se perdeu, não avisa se o cabo foi desconectado, e não tem noção nenhuma de "arquivo". Ele só transporta uma sequência crua de bytes.

O desafio do projeto é construir, por cima desse fio burro, um sistema que consiga:



Fragmentar

Dividir um arquivo grande em pedaços pequenos que cabem no canal.



Confirmar

Garantir que cada pedaço chegou certinho antes de mandar o próximo.



Recuperar

Sobreviver a erros e até a uma desconexão física do cabo no meio da transmissão.



Controlar

Baixar vários arquivos ao mesmo tempo e permitir pausar/retomar/abortar.

ANALOGIA

É como enviar um livro inteiro pelo correio, mas só pode mandar uma carta pequena de cada vez. Cada carta precisa ter um número ("página 4 de 20"), e você só manda a próxima depois que o destinatário confirmar por telefone que recebeu a anterior intacta. É exatamente isso — em bytes — que o protocolo do projeto faz.

2 A solução: um protocolo em camadas

Nada é feito "tudo de uma vez" — cada camada resolve um problema específico

Aplicação (NOVO)

servidor.py / cliente.py — decide "o quê": lista arquivos, escolhe, pausa, salva

Protocolo (NOVO)

protocolo.py — formato do datagrama, checksum, EOP, montar/enviar/receber pacote

Enlace (herdado)

enlace.py / enlaceTx.py / enlaceRx.py — buffers e threads de TX/RX

Física (herdada) — interfaceFisica.py: porta serial UART, 115200 baud

As duas camadas de baixo (Enlace e Física) vieram prontas dos Projetos 1 e 2, sem alteração. O projeto 3 constrói as duas de cima.

A ideia central: as camadas de baixo só sabem mandar e receber *bytes crus*. Quem transforma esses bytes em algo com significado — "isto é um pedaço do arquivo X, pacote 4 de 20" — é a camada de **protocolo**, escrita do zero para este projeto.

3 A unidade básica: o datagrama

Todo pacote trocado entre cliente e servidor tem exatamente essa cara

HEAD — 10 bytes	PAYLOAD — até 100 bytes	EOP — 4 bytes
tipo · fileId · seq · total payloadLen · checksum · rsv	bytes reais do arquivo	0xAA 0x55 0xAA 0x55

Tamanho máximo de um datagrama: $10 + 100 + 4 = 114$ bytes

Campo	Tamanho	Para que serve
tipo	1 byte	Que tipo de mensagem é: HELLO, LISTA, DADOS, ACK, NACK, PAUSAR... (13 tipos ao todo)
fileId	1 byte	A qual arquivo o pacote pertence. 0xFF = mensagem de controle, sem arquivo associado
seq / total	2 + 2 bytes	"Este é o pacote nº seq de total" — é o que permite alternar arquivos sem confundi-los
payloadLen	1 byte	Quantos bytes do payload são realmente válidos (o último pacote de um arquivo raramente enche os 100 bytes)
checksum	2 bytes	Soma de verificação do payload (mesma ideia do checksum do cabeçalho TCP), para detectar bytes corrompidos
EOP	4 bytes	Marca fixa de "fim de pacote" — confirma que o pacote terminou no lugar certo

4 A conversa entre cliente e servidor

Quatro fases, sempre nesta ordem

- 1 Handshake — "você está aí?"**

Cliente manda **HELLO**. Servidor responde **LISTA** com os nomes dos arquivos disponíveis (separados por ;).
- 2 Seleção — "quais arquivos eu quero?"**

Cliente escolhe um nome e manda **SELECIONAR**. Servidor confirma com **CONFIRMAR** e pergunta se quer mais um. Cliente responde **RESPOSTA** (S/N). Esse laço se repete até o cliente dizer "não" — aí o servidor manda **INICIAR** e a transmissão começa.
- 3 Transmissão simultânea — o coração do protocolo**

O servidor não manda um arquivo inteiro de uma vez, nem manda um arquivo por completo antes de começar o outro. Ele intercala: um pacote **DADOS** do arquivo A, espera **ACK** / **NACK**, depois um pacote do arquivo B, espera de novo, volta para A, e assim por diante — até cada arquivo terminar (aí ele sai da rodada e os outros continuam sozinhos).
- 4 Finalização — "terminamos"**

Servidor manda **SUCESSO**. Cliente salva cada arquivo em disco e os dois lados imprimem um resumo: tamanho total e número de pacotes por arquivo.

5 Como o protocolo evita perder ou corromper dados

Quatro mecanismos, cada um cobrindo um tipo diferente de falha

Σ Checksum

Detecta se algum **byte do meio** do payload foi corrompido no fio. Calculado pelo emissor, recalculado pelo receptor — se não bater, o pacote é rejeitado.

🕒 Timeout + reenvio

Se o servidor não recebe resposta em 2 segundos, **reenvia o mesmo pacote**. Não trava, não desiste na primeira tentativa.

✓/✗ ACK / NACK

ACK = "recebi certo, pode mandar o próximo". **NACK** = "algo veio errado (ordem, EOP ou checksum), reenvie o mesmo pacote".

🔄 Resincronização

Se o cabo cai **no meio** da recepção de um pacote, o que sobra no buffer vira lixo. O protocolo descarta bytes até achar a próxima marca EOP e volta a se alinhar sozinho.

ANALOGIA DA RESINCRONIZAÇÃO

Pense em ler um livro cujas páginas se embaralharam no meio da leitura. Em vez de tentar adivinhar onde está o começo da próxima página, você procura o próximo número de página visível (o EOP funciona como esse "marcador") e recomeça a ler dali — sem precisar reiniciar o livro inteiro.

É exatamente esse mecanismo (chamado no código de `resincronizar()`) que permite o requisito mais avançado do projeto: **desconectar fisicamente um dos fios no meio de uma transmissão e reconectar depois** — os dois lados ficam avisando "sem resposta, reenviando..." e, assim que o fio volta, a transmissão continua sozinha de onde parou, sem apertar nenhuma tecla.

6 Controle pelo teclado, durante a transmissão

O cliente fica de olho no teclado sem travar a recepção

Tecla	Mensagem enviada	Efeito
p	PAUSAR	Servidor para de enviar pacotes novos até receber RETOMAR
r	RETOMAR	Servidor reenvia o pacote que ficou pendente e continua normalmente
a	ABORTAR	Os dois lados encerram a comunicação; nenhum arquivo incompleto é salvo como válido

7 O que cada nota exige — e o que foi implementado

Conceito	Exigência do enunciado	Status
C	Transmitir 2 arquivos simultaneamente com sucesso	✓ implementado
B	C + pausar / retomar / abortar pelo teclado	✓ implementado
A	B + reconectar após desconexão física do cabo, sem perder dados	✓ implementado (via timeout + resincronização)
A+	A + documentação com prints do cliente e do servidor	depende da entrega/relatório

RESUMO EM 30 SEGUNDOS, PARA EXPLICAR DE CABEÇA

O Projeto 3 faz dois computadores trocarem arquivos por um fio serial simples, dividindo cada arquivo em pacotes pequenos (datagramas) com um cabeçalho, os dados e uma marca de fim. Cliente e servidor primeiro se cumprimentam e negociam quais arquivos serão baixados; depois o servidor manda os pacotes de todos os arquivos escolhidos alternadamente, sempre esperando confirmação (ACK) antes do próximo — se algo chegar errado ou não chegar a tempo, ele reenvia (NACK / timeout). Um checksum detecta corrupção, uma marca de fim de pacote (EOP) detecta que o pacote chegou inteiro e também serve para realinhar a leitura caso o cabo seja desconectado no meio de um pacote. O usuário ainda pode pausar, retomar ou abortar a qualquer momento pelo teclado. No fim, cada arquivo é salvo intacto e um resumo é impresso dos dois lados.